

Thinking Without Talking: Inference-Time Scaling in Latent Space

DataForge x Pathway Hackathon - "Explain the Frontier"

Interactive Explainer: <https://theshrihari.github.io/Dataforge-Submission/>

Repository: <https://github.com/TheShriHari/Dataforge-Submission>

The Claim

Increasing the number of recurrent latent computation steps at inference time can improve reasoning accuracy without producing any additional natural-language reasoning tokens - but gains diminish, and each extra step costs more compute and time.

That should sound unsurprising at first glance. Of course more computation helps. The non-obvious part is what "more computation" means here: not more parameters, not more tokens, not a bigger model. The exact same 123,713-parameter network, running the exact same weight-shared loop, over the exact same fixed-size hidden tensor. The only variable is K - how many times you spin the loop before reading out the answer. And the accuracy curve that results is steep at first, then nearly flat by $K=10$, then essentially dead by $K=15$. That shape - not the scaling itself but the *saturation structure* - is what this project is built to teach.

Why This Matters: What Standard Architectures Can't Do

A standard feedforward neural network applies its layers exactly once per input. If you want it to "think harder," you have two options: make it bigger, or run it again from scratch on a new input (sampling multiple answers). Both are expensive in different ways.

Large language models extended this with Chain-of-Thought (CoT) prompting: the model writes out its intermediate reasoning as natural language tokens, which then become part of the context for subsequent steps. That works, but it carries a structural cost. Every emitted token appends to the sequence, expanding the KV-cache. The attention computation over a sequence of length N costs $O(N^2)$ in memory. More critically, the reasoning is constrained by syntax: a grammatical slip or early hallucination propagates forward because the model's next step conditions on the garbled text. Intermediate reasoning tokens also leak information about reasoning strategy in ways that may not correspond to how the model is actually computing.

Recurrent latent iteration sidesteps these issues by keeping all intermediate reasoning *inside* the hidden state - no tokens emitted, no context window expansion, no syntactic requirements. The state tensor $\mathbf{h}$ stays the same shape throughout: $\mathbb{R}^{(C \times H \times W)}$ for spatial tasks, or equivalently a fixed-dimensional vector for sequence tasks. Each iteration refines it. The final readout happens after K loops. From the outside, the model still produces one output, same as always.

The cost you pay is different: latency scales linearly with K (you run the loop K times), and there's no external transcript of what the model "decided" at each step. Interpretability requires probing the hidden state directly, which is non-trivial.

The Mechanism: How a Weight-Shared Loop Propagates Information

Our implementation uses a 2D maze pathfinding task, which is pedagogically useful because the computation the model needs to do - trace a path from a start cell to a goal cell through open corridors - has a known structure that maps cleanly onto receptive field expansion.

Architecture

The full model (`m1/model.py`) has three components:

1. Input Encoder: A 1×1 pointwise convolution mapping a 3-channel input tensor (wall mask, start cell, goal cell) into a 48-dimensional latent tensor of shape $\mathbb{R}^{(48 \times 15 \times 15)}$. The 1×1 kernel is deliberate: it has zero spatial receptive field, meaning the model cannot see neighboring cells at this stage. Solving the maze requires information to flow across cells, and that can only happen inside the recurrent loop.

2. Shared Convolutional GRU Core (`ConvGRURecurrentCore`): Applied K times, with identical weights every iteration. The update equations as implemented in `m1/model.py` :

```
gates = sigmoid(gate_h(h_prev) + gate_x(h_init))
r, z  = chunk(gates, 2, dim=1)          # reset gate, update gate
cand  = tanh(cand_h(r * h_prev) + cand_x(h_init))
h_k   = (1 - z) * h_prev + z * cand
```

`gate_h` and `cand_h` use 3×3 convolutions with `padding=1`, so each application expands the effective receptive field by 2 grid cells (1 in each direction). `gate_x` and `cand_x` are 1×1 convolutions that re-inject the original encoded input *hinit* at every step - this lets the model keep the maze layout in view regardless of how much *hprev* has evolved.

The update gate z is the key to the saturation curve. A z near 1 applies the candidate aggressively; a z near 0 preserves the prior state. The model learns to gate conservatively once it has a good solution estimate - which is mechanically why the accuracy curve flattens. The gating also prevents the hidden state from exploding over large K , making the architecture stable at $K=20$ even though training only went to $K=15$.

3. Readout Head: A 1×1 convolution projecting the final h_K into a scalar probability per cell, followed by sigmoid. Threshold at 0.5 to get a binary path prediction.

Training

Trained (`m1/train.py`) over 50 epochs on 500 synthetic 15×15 mazes generated by a randomized DFS backtracker. Per-batch effort K was sampled as `np.random.randint(1, 16)` - $K \in \{1, 2, \dots, 15\}$ uniformly. Deep supervision was applied: the loss was computed at every intermediate step $k=1..K$ and averaged, not only at the final step. Without this, the model would converge to a good output at the sampled K but produce near-random outputs at other K values, which would defeat the purpose of showing a smooth scaling curve.

Optimizer: AdamW ($\text{lr}=0.003$, $\text{weight_decay}=1\text{e-}4$) with cosine annealing to $\text{lr}=1\text{e-}5$. Loss: Binary Cross-Entropy + $0.5 \times \text{soft IoU loss}$, averaged across all intermediate steps.

What the Artifact Shows You

The interactive web explainer has four modules. To be explicit about what is and isn't live computation:

Module	What it shows	Provenance
Reasoning Effort Slider (Module 1)	Maze grid with predicted path highlighted at each K	PRECOMPUTED - inference outputs for K=1..20 per maze stored in <code>site/data/example_traces.json</code> . No neural network runs in-browser.
Auto Button (Module 1)	Steps K upward and halts when the model reaches a fully connected path (typically K=4–5)	PRECOMPUTED playback; the halting condition is evaluated client-side on the stored mask
Empirical Scaling Chart (Module 2)	Exact solve rate % and Path IoU % vs. K across 100 mazes	PRECOMPUTED - run by <code>m1/evaluate.py</code> on 100 held-out test mazes (seed=999), stored in <code>results/accuracy_by_effort.json</code>
Individual Puzzle Trace (Module 2)	Per-step progress curve for one selected maze	PRECOMPUTED from <code>site/data/example_traces.json</code>
BDH-CQ Comparison Cards (Module 3)	ARC-AGI-1 pass@2 at Low/Medium/High effort	EXTERNAL - reproduced from arXiv:2608.09888 Table 5
Architectural Comparison (Module 4)	CoT vs. latent recurrence side-by-side	ILLUSTRATIVE - explanatory text, not data
UI interactions	Slider, view switching, row highlighting	LIVE client-side JavaScript

The Auto button is worth pausing on. It doesn't play through K=1 to K=20 as a reel. It steps K upward one at a time and stops the moment the model's predicted path becomes fully connected from start to goal. For most test mazes that happens at K=4 or K=5. You watch the model click into a full solution and stop - which is a more visceral demonstration of the diminishing-returns thesis than the aggregate chart, because it shows the exact moment where more effort would be wasted.

The Numbers: What We Actually Measured

All results below are **own toy implementation, precomputed on a 100-maze held-out test set (seed=999), not independently audited**. Source: `results/accuracy_by_effort.json` .

K	Exact Solve %	Path IoU %	Latency (ms/maze)
1	1.0%	55.94%	0.666
3	19.0%	76.63%	1.747
5	44.0%	87.69%	2.911
7	63.0%	93.12%	4.098
10	81.0%	96.61%	6.155
12	84.0%	97.42%	7.849
15	88.0%	98.01%	9.374
16	86.0%*	98.12%	9.059
20	88.0%	98.13%	12.884

**Note on K=16: The slight dip to 86.0% reflects mild out-of-distribution drift beyond the $K \in [1, 15]$ training curriculum and sample variance on 100 mazes; Path IoU remains monotonic (98.12%).*

From K=1 to K=10: exact solve rate rises from 1% to 81% - an 80-point absolute gain. From K=10 to K=20: 81% to 88% - a 7-point gain - while latency more than doubles (6.155 ms $\rightarrow$ 12.884 ms). The inflection point around K=10 is the empirical signature of saturation.

Path IoU captures partial credit: even at K=1, IoU of 55.94% means the model is already orienting toward the correct corridor direction, even though it can't bridge the full path. By K=10 the IoU is 96.61%, meaning predicted and ground-truth paths nearly coincide, even though 19% of mazes still fail the strict binary exact-solve test (the model overshoots or misses a single cell).

Training horizon: K=16–20 are out-of-distribution. Training used $K \in \{1..15\}$ (`np.random.randint(1, 16)` in `ml/train.py:78`). The model generalizes because the ConvGRU core is weight-shared with no explicit step-count parameter - the same function is applied regardless of iteration count. But this is extrapolation. The slight non-monotonicity at K=16 (86% solve rate, down from 88% at K=15) is consistent with mild distributional drift and should not be over-interpreted on a 100-maze sample.

The BDH Connection: Why This Scales to Frontier Systems

The maze experiment isolates K as the single variable in a domain with exact ground truth. The concept scales to frontier language models, and Pathway's work provides the most direct published example.

BDH vs. BDH-CQ: A Necessary Distinction

The Dragon Hatchling (BDH) (Kosowski et al., 2025, arXiv:2509.26507) is a foundational architecture proposal: a biologically-plausible recurrent system that establishes latent-state updates and non-autoregressive parallel processing as the basis for general sequence modeling. BDH is the architectural family.

BDH-CQ (Kosowski et al., 2026, arXiv:2608.09888) is a specific system built within the BDH family, adding "Context Queries" - in-context demonstration learning. The feature directly relevant here is *effort modulation*: BDH-CQ can be run at LOW, MEDIUM, or HIGH compute budgets at inference time, without emitting scratchpad tokens between the effort levels. This is the frontier-scale analog of our K parameter.

The published evaluation is on ARC-AGI-1. **Evidence type: developer-reported; the internal LOW/MEDIUM/HIGH scaling ablation (Table 5) is developer-reported only and not independently reproduced.**

Effort Level	ARC-AGI-1 Pass@2	Hardware Cost vs. HIGH
LOW	21.0%	78%
MEDIUM	27.0%	89%
HIGH	29.5%	100%

The structural parallel is direct. LOW→MEDIUM gains 6.5 points for 11% more compute. MEDIUM→HIGH gains 2.5 points for another 11% compute. The marginal return drops by more than half. The same saturation shape appears in our 123K-parameter maze solver.

What changes inside BDH-CQ when effort is increased: the recurrent latent state - a continuous hidden representation that is never decoded into text between steps - runs more update iterations. The trained synaptic weights do not change. The parameter count does not change. Only the latent state trajectory is extended, the same as increasing K in our model.

Limitations

These are specific, not generic:

- Domain gap:** Maze pathfinding has polynomial-time optimal solutions and clean binary ground truth. Whether the saturation curve shape holds for open-ended reasoning tasks (mathematics, code, natural language) is an open question, not a given.
- Scale extrapolation:** Our model is 123,713 parameters. BDH-CQ is approximately 150M (developer-reported). Effects visible at toy scale may qualitatively shift at language model scale - the saturation point might move, the out-of-distribution generalization might not hold, or the gating dynamics might differ.
- Interpretability gap:** When the maze solver fails at K=4, we can visualize the predicted path and see exactly where it breaks. For a frontier recurrent latent model, diagnosing failure at a given effort level requires probing classifiers over continuous hidden states - there is no reasoning transcript to read. This is a real deployment limitation.
- The K=16–20 behavior is not guaranteed:** Our training distribution ends at K=15. The model happens to generalize, probably because gated weight-sharing has a naturally stationary fixed-point structure. A different architecture or a harder task might not generalize.
- Test set size:** 100 mazes is sufficient to show the scaling trend clearly but too small to make strong statistical claims about exact solve rates - the 88% figure has a standard 95% Wald margin of error of $\pm 1.96 \sqrt{\frac{0.88 \times 0.12}{100}} \approx \pm 6.4\%$ (81.6% to 94.4%), while an asymmetric Wilson score interval gives 80.2% to 93.1%. Do not over-interpret the specific point estimates; interpret the overall saturation shape.

What You Should Be Able to Explain After Using This

Q: Why does K=1 give 1% solve rate but K=5 gives 44%, even though the model weights don't change?

Each recurrent loop applies a 3×3 convolution, expanding the effective receptive field by 2 cells per iteration. At $K=1$, any given cell's output only integrates information from its immediate neighbors - which can't bridge a 15×15 path. By $K=5$, the effective reach is 10 cells from any starting point, enough to span most maze corridors.

Q: Why does K=20 barely outperform K=10?

The update gate z in the GRU learns to suppress updates once the hidden state has converged to a stable solution estimate. Further loops apply small corrections to an already-confident representation. This is mechanically why the curve flattens - not because the model hits a computational wall, but because the gating function has learned to disengage.

Q: What changes inside BDH-CQ at HIGH vs. LOW effort, and what doesn't?

The recurrent latent state runs more update iterations. The trained synaptic weights don't change. The parameter count doesn't change. The output format doesn't change. Only the hidden state trajectory is extended.

Q: What's wrong with calling CoT and latent recurrence "two ways to do the same thing"?

CoT tokens enter the context window and are re-processed by attention in subsequent steps - creating $O(N^2)$ memory cost, syntactic constraints, and hallucination propagation risk. Latent recurrence updates a fixed-shape tensor with no sequence growth and no emitted text. The information is stored and processed differently at every level.

Closing

A 123,713-parameter model, using no additional parameters and no intermediate text, improves its exact-solve rate from 1% to 81% purely by repeating the same computation - but the last 7 percentage points (up to 88%) cost twice the compute. That curve shape is the empirical signature of a recurrent system working through a constraint-propagation problem: rapid early progress as the receptive field spans the maze, then diminishing returns as the hidden state saturates.

Pathway's BDH-CQ results on ARC-AGI-1 show the same structure at a different scale, on a harder task, at developer-reported confidence. The toy and the frontier are doing structurally the same thing. This artifact exists to make that structure visible.

Primary Sources

1. Kosowski et al. (2025). *The Dragon Hatchling: The Missing Link Between the Transformer and Models of the Brain*. [arXiv:2509.26507](https://arxiv.org/abs/2509.26507) - BDH architecture establishing recurrent latent state updates.
2. Kosowski et al. (2026). *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. [arXiv:2608.09888](https://arxiv.org/abs/2608.09888) - BDH-CQ effort modulation on ARC-AGI-1 (Table 5, developer-reported).
3. Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. [arXiv:2408.03314](https://arxiv.org/abs/2408.03314) - Formalizes test-time compute vs. parameter scaling trade-offs.
4. Goyal et al. (2024). *Think before you speak: Training Language Models With Pause Tokens*. ICLR 2024. [arXiv:2310.02226](https://arxiv.org/abs/2310.02226) - Motivates non-verbalized computational pauses.
5. Ballas et al. (2015). *Delving Deeper into Convolutional Networks with Shape-Preserving Recurrent Models*. [arXiv:1511.06432](https://arxiv.org/abs/1511.06432) - Original ConvGRU formulation; `m1/model.py` is an independent from-scratch

implementation of the same equations.

AI Assistance Disclosure: Code scaffolding, unit test generation, SVG markup, and documentation drafting used Antigravity (Google DeepMind). All model architecture design, training logic, evaluation harnesses, and mathematical claims were validated by the team with test-driven verification. All numbers in this document are sourced from `results/accuracy_by_effort.json` and `site/data/bdh_cq_reference.json` (reproducing Table 5 of arXiv:2608.09888).